

UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS ARARANGUÁ
GRADUAÇÃO EM TECNOLOGIA DA INFORMAÇÃO E COMUNICAÇÃO

Ivan Marcos Cristiano Júnior & José Vitor Francisco Fernandes

Trabalho Prático – Unidade II

Manutenção e Evolução de um Projeto Open Source – Micropolis

Araranguá
2026

1. Identificação do Projeto e Justificativa da Escolha

O objetivo do presente trabalho consiste em aplicar, de forma integrada e rastreável por controle de versão Git, os processos de compreensão arquitetural, diagnóstico de problemas de manutenibilidade, refatoração estrutural e manutenção perfectiva sobre um software de código aberto. O sistema selecionado como objeto de estudo é o Micropolis, derivado do projeto MicropolisJ (originalmente concebido por Jason Long como uma conversão para Java do código-fonte aberto do SimCity clássico) e disponibilizado sob a licença pública geral GNU GPLv3. O código-fonte original que serviu de ponto de partida encontra-se hospedado em um repositório público.

A escolha do projeto Micropolis justifica-se pelo atendimento integral aos critérios técnicos e metodológicos estabelecidos na Etapa 1 do trabalho prático. Trata-se de uma aplicação desktop completa e interativa de simulação urbana, não de uma biblioteca ou framework abstrato, desenvolvida em linguagem Java com interface gráfica em Swing/AWT, gerenciada pelo Apache Maven e provida de suíte de testes automatizados em JUnit 5 e AssertJ. Seu porte médio oferece complexidade arquitetural suficiente para a análise de problemas reais de manutenibilidade sem depender de serviços externos ou bancos de dados. Ademais, por resultar de uma migração histórica de C para Java, o código apresenta code smells clássicos propícios à aplicação prática de refatorações e evolução funcional.

Critério de Seleção	**Verificação no Projeto Micropolis**
Código aberto	Repositório público hospedado no GitHub e licenciado sob GNU GPLv3.
Tecnologia	Desenvolvido em Java com interface gráfica em Swing/AWT, automação de build em Apache Maven e suíte de testes em JUnit 5 e AssertJ, tecnologias de pleno domínio da equipe
Tipo de software	Aplicação desktop completa (game).
Tamanho	Porte médio (aproximadamente 60 classes Java e 13.400 linhas de código na baseline).
Documentação	Contém arquivo README.md com pré-requisitos, instruções de compilação, execução, divisão de pacotes e especificação do formato binário.
Dependências	Declaradas no pom.xml (JUnit Jupiter e AssertJ) e resolvidas automaticamente pelo Maven, sem dependência de bancos de dados externos, serviços em nuvem ou APIs de terceiros.
Build e Execução	Compila e empacota localmente sem erros, gerando um artefato JAR executável autossuficiente.
Testes	Possui suíte de testes unitários automatizados em JUnit 5 (22 testes na baseline, distribuídos em 3 classes) e configuração de cobertura com JaCoCo.
Histórico	Histórico Git ativo de commits, possibilitando o rastreamento de alterações anteriores.
Manutenibilidade e Evolução	Por se tratar de código historicamente portado de C para Java, apresenta code smells clássicos (Large Class, Long Method, Duplicated Code, Conditional Complexity e Feature Envy).

2. Caracterização e Compreensão Inicial do Sistema

2.1 Objetivo do Software

O software tem como objetivo simular a construção, o planejamento urbano e a gestão econômico-financeira de uma cidade em tempo real. O jogador define zonas residenciais, comerciais e industriais, constrói redes de energia elétrica e transporte (estradas, ferrovias, linhas de transmissão, porto e aeroporto), implanta serviços públicos (delegacias de polícia, corpo de bombeiros, hospitais e estádio) e administra alíquotas de impostos, orçamento anual, índices de poluição, criminalidade e ocorrência de desastres.

2.2 Linguagem e Tecnologias

Java, interface gráfica Swing/AWT (javax.swing, java.awt), build no Apache Maven (pom.xml) e testes automatizados em JUnit 5 e AssertJ, com cobertura via JaCoCo. Ele é compilado por meio do comando mvn clean package. Para rodar apenas os testes, executa-se mvn test.

2.3 Organização do Projeto

Segue a estrutura padrão do Maven:

- src/main/java/micropolisj/engine: Motor de simulação da cidade, desacoplado da interface gráfica. É aqui onde se encontra o código principal.
- src/main/java/micropolisj/gui: Componentes visuais em Swing (janela principal, mapa, minimapa e diálogos).
- src/main/java/micropolisj/build_tool: Utilitários auxiliares para geração de imagens de tiles.
- src/main/resources: Definições de tiles (graphics/tiles.rc), imagens (.png), sons (.wav) e textos de tradução (strings/*.properties).
- src/test/java/micropolisj/engine: Onde se encontram os testes unitários do jogo.

3. Baseline

Antes de iniciar qualquer alteração, o projeto original foi compilado, testado e executado para registrar a baseline:

Informação	Resultado
Commit da baseline	9f847f9 (Add unit tests for spend method in Micropolis class)
URL do projeto original	https://github.com/dheid/micropolis.git
Build – mvn clean package	Sucesso
Quantidade de testes	22 testes
Testes executados com sucesso	22/22
Execução da aplicação	Sucesso

4. Problemas de Manutenibilidade Identificados

A refatoração é uma alteração na estrutura interna do código que melhora qualidades não-funcionais (simplicidade, clareza e flexibilidade) sem alterar seu comportamento funcional, tendo como objetivo reduzir o débito técnico. Conforme os critérios da Etapa 4, foram identificados quatro problemas de manutenibilidade que dificultam a compreensão, a evolução e os testes do sistema:

	Micropolis.java	RoadLikeTool.java	OverlayMapView.java	MapGenerator.java
Code Smell	Large Class, Long Method e Feature Envy.	Duplicated Code, Long Method, Conditional Complexity e Comments.	Duplicated Code, Long Method e Números Mágicos.	Duplicated Code, Long Method, Conditional Complexity e Variable Shadowing.
Localização	Micropolis.java (2615 linhas totais).	RoadLikeTool.java: métodos layRail layRoad e layWire (L329–428).	OverlayMapView.java: métodos draw*, check* e paintComponent.	MapGenerator.java: métodos doBRiv/doSRiv e smoothRiver/smoothTrees.
Evidência	Mais de 80 atributos e 120 métodos misturando simulação, orçamento, censo, energia, mapas de crime/poluição, sprites, desastres, notificações e gravação de arquivos .cty.	12 blocos quase idênticos (// check east..north) para construir pontes, túneis e cabos sobre água (~150 linhas), além de custo de demolição triplicado.	6 métodos draw* com o mesmo laço for duplo; 3 métodos de overlay repetindo laços com img.setRGB(...); e paintComponent com 100 linhas e retorno mágico -1.	doBRiv() e doSRiv() possuem código idêntico (mudando apenas a matriz BRMatrix 9x9 vs. SRMatrix 6x6); smoothRiver e smoothTrees repetem o cálculo de 4 vizinhos e ocultam os atributos mapX/mapY.
Dívida Técnica	Adicionar qualquer nova construção exige editar dezenas de trechos dentro de um arquivo de 2.615 linhas, dificultando testes isolados e aumentando o risco de erros.	Alterar regras de travessia de água ou de limpeza de terreno exige atualizar 3 métodos e 12 blocos manualmente.	Dificulta a leitura do ciclo de pintura do Swing e a manutenção das camadas de visualização do minimapa.	Exige manutenção duplicada na geração de rios e florestas, e o sombreamento de mapX/mapY pode causar bugs no cursor global do gerador.
Tratamento	Extract Class, Move Method e Encapsulate Field.	Extract Method e Introduce Parameter Object.	Extract Method parametrizado e constantes simbólicas.	Extract Method parametrizado e renomeação de variáveis locais.

5. Refatoração Realizada

As refatorações seguiram o fluxo incremental visto em aula: identificar o problema, apoiar-se nos testes da baseline (commit 9f847f9), aplicar a técnica de refatoração em passos pequenos, rodar os testes novamente e registrar o commit.

5.1 Decomposição de Micropolis.java (Commits 8f5ad77, ef2f4b2, aca9aa6 e 2899af8)

Aplicou-se Extract Class, Move Method e Encapsulate Field (commits 8f5ad77, ef2f4b2, aca9aa6 e 2899af8) para dividir Micropolis.java em 8 novas classes no pacote micropolisj.engine (que, junto a CityEval.java, somam 9 colaboradores): BudgetManager.java (orçamento e impostos, commit 2899af8), CensusManager.java (contadores de censo e demanda RCI, commit 2899af8), PowerManager.java (propagação de energia, commit 2899af8), MapAnalysis.java (mapas de poluição, crime, tráfego e valor da terra, commit 2899af8), SpriteManager.java (veículos e explosões, commit 2899af8), CityNotificationManager.java (mensagens, sons e listeners, commit 2899af8), DisasterManager.java (desastres, commits 8f5ad77 e ef2f4b2) e CityPersistence.java (arquivos .cty, commits 8f5ad77 e aca9aa6).

Após a refatoração, Micropolis.java diminuiu de 2.615 para 1.165 linhas, passando a apenas coordenar a simulação e delegar o trabalho para classes menores e coesas, sem quebrar a interface pública.

5.2 Eliminação de Duplicação em RoadLikeTool.java (Commit 35d4083)

Aplicou-se Introduce Parameter Object (commit 35d4083) para criar o record WaterCrossing (com as constantes BRIDGE, TUNNEL e UNDERWATER_WIRE) e Extract Method para criar layAcrossWater, clearingCost e métodos booleanos com nomes claros (bridgeReachesEast, tunnelReachesNorthOrSouth, etc.).

Como resultado, houve a eliminação dos 12 blocos repetidos de verificação direcional e os comentários // check east..north, centralizando a regra de travessia de água em um único método.

5.3. Refatoração de OverlayMapView.java (Commit 9fd895d)

Aplicou-se Extract Method (commit 9fd895d) para extrair o método genérico drawGrid (usado pelos 6 métodos draw*), além de dividir paintComponent (de 100 para 16 linhas) em paintTiles, paintOverlay e paintViewRects, criando as constantes ALREADY_DRAWN = -1, CELL_SIZE_SMALL = 6 e CELL_SIZE_LARGE = 24.

Como resultado, os laços duplicados foram removidos e o método de renderização do Swing ficou mais curto e direto.

5.4. Unificação em MapGenerator.java (Commit 627fc13)

Aplicou-se Extract Method (commit 627fc13) para unificar doBRiv() e doSRiv() no método doRiver(char[][] riverMatrix) e extrair compute4NeighborMask (usado em smoothRiver e smoothTrees), além de renomear as variáveis locais mapX/mapY para x/y.

Como resultado da refatoração, eliminou-se a duplicação na criação de rios e na suavização do mapa e removeu-se o sombreamento dos atributos da classe, mantendo-se o comportamento.

6. Solicitação de Manutenção Perfectiva

Para a Etapa 6, foi proposta a adição do edifício Universidade (University) ao jogo:

- Descrição da necessidade: O jogo original possui delegacias, bombeiros, estádio, porto e aeroporto, mas não conta com uma instituição de ensino superior capaz de estimular o crescimento urbano ao seu redor, aumentar o valor da cidade e melhorar a pontuação do prefeito.
- Comportamento esperado:
    - Adicionar o botão University (MicropolisTool.UNIVERSITY) na barra de ferramentas para construir um campus de 4x4 tiles pelo custo de $3.000.
    - Durante a simulação (MapScanner), contabilizar cada universidade no censo (universityCount), verificar se há energia elétrica (checkZonePower), reparar seus 16 tiles a cada 16 ciclos e, quando energizada, aplicar um bônus de crescimento local (adjustROG(2)) na região a cada 8 ciclos.
    - Na avaliação da cidade (CityEval), somar $4.000.000 ao valor patrimonial da cidade (cityAssValue) por universidade e adicionar +15 pontos no cityScore (limitado ao máximo de +50 pontos).
    - Exibir o nome "University" (zone.27) ao inspecionar o edifício com a ferramenta Query.
- Regras e restrições: Ocupa área fixa de 4x4 tiles (IDs 960 a 975, com centro em UNIVERSITY = 965); exige saldo mínimo de $3.000 e terreno livre/demolível; conduz eletricidade (conducts) e está sujeita a incêndios e desastres (isArsonable, isVulnerable).
- Critérios de verificação: Construir a universidade com saldo de $10.000 deve retornar ToolResult.SUCCESS, reduzir o saldo para $7.000 e posicionar os tiles 960 a 975; após um ciclo completo de varredura (animate()), getUniversityCount() deve retornar 1; e o botão e o edifício 64x64 devem aparecer corretamente na interface gráfica.

7. Análise de Impacto (Previsto)

Antes da implementação (Etapa 7), previram-se os seguintes pontos de modificação:

- Pacotes afetados: micropolisj.engine, micropolisj.gui, src/main/resources e src/test/java/micropolisj/engine.
- Classes e métodos: Behavior.java, TileConstants.java, MicropolisTool.java (beginStroke), BuildingTool.java (applyZone), MapScanner.java (processTile), Micropolis.java (initTileBehaviors), CensusManager.java (clearCensus), CityEval.java (assessValue, doScoreCard) e MainWindow.java (makeToolbar).
- Dados, recursos e testes: Inclusão dos tiles 960..975 em graphics/tiles.rc, textos em GuiStrings.properties e StatusMessages.properties, imagens PNG do ícone e do edifício, e novos testes em MicropolisTest.java.

8. Descrição da Implementação

- Constantes e comportamento (Behavior.java e TileConstants.java -- commits bacbada e d40387c): Adicionado UNIVERSITY ao enum Behavior (commit bacbada) e criadas as constantes UNIVERSITY_BASE = 960, UNIVERSITY = 965 e UNIVERSITY_LAST = 975 em TileConstants.java, atualizando também isArsonable() e isVulnerable() (commit d40387c).
- Ferramenta de construção (MicropolisTool.java e BuildingTool.java -- commits 960b2e2 e 6a9e035): Criada a ferramenta UNIVERSITY(4, 3000) (commit 960b2e2) e adicionado o case UNIVERSITY: em BuildingTool chamando applyZone(eff, UNIVERSITY) (commit 6a9e035).
- Simulação e censo (MapScanner.java, Micropolis.java e CensusManager.java -- commits 1aff5b4, 8729630 e 2899af8): Registrado o comportamento "UNIVERSITY" (commit 8729630), criado o contador universityCount em CensusManager (commit 2899af8) e implementado doUniversity() em MapScanner.java (commit 1aff5b4).
- Avaliação da cidade (CityEval.java -- commit ea54d3f): Adicionado engine.getUniversityCount() * 4000 (equivalente a $4.000.000) em assessValue() e Math.min(50, engine.getUniversityCount() * 15) em doScoreCard().
- Interface gráfica e recursos (MainWindow.java, TileImages.java, tiles.rc, .properties e .png -- commits aca8f60 a b61ba4f): Definidos os tiles 960 a 975 em graphics/tiles.rc, adicionado o botão na barra de ferramentas da MainWindow, configurado o corte dos 16 sub-tiles da imagem 64x64 em TileImages.java, incluídas as strings em GuiStrings.properties e StatusMessages.properties e adicionados os ícones/sprites (icuniv.png, icunivhi.png e university.png).

9. Testes e Evidências de Regressão

Na Etapa 9, foram adicionados 6 novos testes unitários em MicropolisTest.java (commits bb855a3, 2899af8 e 253ebe8) (2 para validar a Universidade e 4 para validar os gerenciadores extraídos na refatoração), aumentando a suíte de 22 para 28 testes:

- testBuildUniversitySpends3000AndPlaces4x4Zone() (commit bb855a3): verifica que construir a universidade retorna ToolResult.SUCCESS, desconta $3.000 e posiciona os tiles 960 a 975.
- testUniversityCountAndEvaluationAssessedValue() (commit bb855a3): verifica que, após 22 passos de animate(), getUniversityCount() passa a ser 1.
- testExtractedCollaboratorsAreInitializedAndCohesive(), testBudgetManagerGeneratesBudgetAndTracksTaxRate(), testMapAnalysisQueriesZoneStatusAndLandValue() e testSpriteManagerSpawnsAndMovesExplosionSprite() (commits 2899af8 e 253ebe8): testam a inicialização e o funcionamento dos gerenciadores extraídos.

Classe de Teste	Baseline	Repositório do grupo	Status
CityBudgetTest.java	2	2	100%
IsTileDozeableTest.java	4	4	100%
MicropolisTest.java	16	22	100%
Total	22 testes	28 testes	100% - 0 regressões

10. Comparação entre Impacto Previsto e Impacto Observado

Escopo	Impacto Previsto	Impacto Observado	Diferenças Observadas
engine	7 arquivos (Behavior, TileConstants, MicropolisTool, BuildingTool, MapScanner, Micropolis, CityEval).	8 arquivos (os 7 previstos + CensusManager.java).	Com a refatoração de Micropolis (commit 2899af8), o contador universityCount ficou em CensusManager.java. Em TileConstants.java (commit d40387c), também foi preciso atualizar isArsonable() e isVulnerable().
gui	1 arquivo (MainWindow.java).	2 arquivos (MainWindow.java e TileImages.java).	O arquivo tiles.png original só ia até o tile 959; por isso, foi necessário alterar TileImages.java (commit 50f7f29) para montar os sub-tiles 960..975 da Universidade.
resources	5 arquivos (tiles.rc, 2 .properties, 1 sprite e 1 ícone).	7 arquivos (tiles.rc, 2 .properties, icuniv.png, icunivhi.png, university.png na raiz e em graphics/).	O botão da MainWindow exige dois ícones (normal e selecionado), além de university.png em dois diretórios para carregamento pelo ClassLoader.
test	1 arquivo (MicropolisTest.java).	1 arquivo (MicropolisTest.java).	Ocorreu exatamente como previsto, sem quebrar os 22 testes originais.

11. Reflexão sobre as Dificuldades ao Modificar Software de Terceiros

A manutenção do projeto Micropolis evidenciou desafios práticos comuns em sistemas desenvolvidos por terceiros, a começar pelo forte acoplamento entre código-fonte e recursos externos, visto que a adição dos tiles 960..975 da Universidade exigiu sincronizar alterações manuais entre o ficheiro graphics/tiles.rc, os intervalos numéricos em TileConstants.java e o carregamento de imagens em TileImages.java. Esse cenário confirmou na prática a importância da refatoração antes da evolução funcional: dividir a classe Micropolis.java em nove gerenciadores de domínio (commits 8f5ad77 e 2899af8) reduziu o ficheiro principal para metade, facilitou a compreensão da arquitetura e isolou os pontos exatos onde o censo e a avaliação da cidade precisavam de ser modificados.

Além disso, a herança do código original em C dificultou a criação dos testes unitários, pois o motor utiliza máscaras de bits e divide a simulação em 16 fases, sendo necessário identificar que o método animate() só avança a simulação a cada duas chamadas e exige pelo menos 22 execuções seguidas para concluir a leitura do mapa e atualizar o censo. Por fim, lidou-se com restrições do próprio ambiente de build, em que o plugin spotless-maven-plugin configurado no pom.xml original bloqueava a compilação por mínimas diferenças de espaçamento, exigindo o seu ajuste (commit 38616f5) para viabilizar o desenvolvimento e a execução contínua dos testes.

REFERÊNCIAS

BORDIN, Andréa. **Refatoração: Melhorando a Qualidade de Código Pré-Existente**. Notas de aula da disciplina de Engenharia de Software II. Araranguá: UFSC, 2026

FOWLER, Martin. **Refactoring: Improving the Design of Existing Code**. 2. ed. Boston: Addison-Wesley, 2020.

Repositório Original:

https://github.com/dheid/micropolis

Repositório do Grupo:

https://github.com/Koenegus/micropolis-polished